

BugTrail

Engineering reference. A bug report goes in; a named pull request, its author, the suspect diff, the owning team, and a drafted failing regression test come out.

Engineering Reference

Python 3.9+ · stdlib only

42 tests passing

Deterministic · no model in the path

A bug report goes in. A named pull request, its author, the suspect diff, the owning team, and a **proven** failing regression test come out.

Repository: [Monish-WBD/marrow-bugtrail-fixture](#) (public) **Stack:** Python 3.9+ standard library only, plus [git](#). No dependencies, no network, no credentials.

Table of contents

1. [The problem](#)
2. [What BugTrail does](#)
3. [Quick start](#)
4. [How to test everything](#)
5. [How it works](#)
6. [What we proved](#)
7. [Key design decisions](#)
8. [Bugs we hit and fixed](#)
9. [Repository layout](#)
10. [Path to production — Jira](#)
11. [Production hardening checklist](#)
12. [Known limitations](#)
13. [Work split and next steps](#)

1. The problem

When a bug is filed, someone has to answer two questions:

Which change caused this? and **who should look at it?**

CodeSage already answers the first half. It posts an AI triage comment suggesting priority, severity, component, team, and a *starting-point file*.

What it does **not** do is close the loop to source control. An engineer still has to:

1. Run `git log` on the suggested file
2. Read through recent commits
3. Work out which PR each commit came from
4. Check who authored it
5. Decide which one is plausibly related
6. Write a test that reproduces the bug

That is **10–20 minutes of mechanical archaeology per bug**, repeated for every bug, on every platform.

BugTrail automates exactly that gap. It *consumes* CodeSage's output rather than replacing it.

2. What BugTrail does

Does

CAPABILITY	DETAIL
Parse triage input	Reads a CodeSage comment deterministically; handles Jira's ADF format
Walk history	Follows the seed file through renames, plus its module siblings
Map commit → PR	Handles squash merges, merge commits, and direct pushes with no PR
Exclude the impossible	Bots, reverts, generated sources, anything landing after the report
Rank and explain	Four weighted signals, every score explained in plain English
Resolve ownership	CODEOWNERS lookup, and flags disagreement with CodeSage's suggested team
Draft a regression test	Correct path, framework, class name and symbol, derived from the suspect diff
Prove the attribution	Compiles the test at two revisions to confirm behaviour changed at that commit

Does not


```

1. PR #11 "Fix preroll marker propagation to overlays (#11)"
   commit f10254a (squash merge)
   author Bob Sharma <bob@example.com>
   landed 2026-07-01 13:40
   score 0.82 (4 line(s) changed)
   why
       - modifies the file CodeSage pointed to
       - changed code mentions 'skip', 'marker', 'overlay', 'preroll'
       - landed the same day the bug was reported

   suspect change
       - fun markersForOverlay(): List<TimelineMarker> = processor.skippableMarkers()
       + fun markersForOverlay(): List<TimelineMarker> =
       +     processor.skippableMarkers().filterNot { it.isPreroll }

```

Step 2 — proving the drafted test isolates that PR:

```

at HEAD (bug present, expect FAIL)
  FAIL a skippable preroll was reported as not skippable
  => as expected: the test catches the bug

at 5ccb5dc^ (before the suspect PR, expect PASS)
  PASS a skippable preroll remains skippable on the ad-free tier
  => as expected: the behaviour was intact here

CONFIRMED: behaviour changed at the suspect commit. Attribution holds.

```

4. How to test everything

All commands run from the repository root. Nothing needs a network.

4.1 Full report

```
python3 tools/bugtrail/cli.py --comment fixtures/triage/SYN-002.txt --report
```

BASH

****Expect:**** bug details, ranked suspects with authors and reasons, suspect diff, CODEOWNERS, drafted test.

4.2 Prove the regression test isolates the suspect

BASH

```
tools/bugtrail/verify/verify.sh
```

Expect: `FAIL` at HEAD, `PASS` before the suspect commit, ending `CONFIRMED`. Requires `swiftc`.

4.3 Score the ranker against known culprits

BASH

```
python3 tools/bugtrail/cli.py --eval
```

```
bug      expected  predicted  P@1  P@3
SYN-001  #5           5, 2, 4    yes  yes
SYN-002  #11          11, 3      yes  yes
SYN-003  #4           4, 5, 2    yes  yes
precision@1: 3/3 (100%)  precision@3: 3/3 (100%)
```

4.4 Derive ground truth from git history, then score against it

BASH

```
python3 tools/bugtrail/mine_szz.py --repo . --scan 200 --out fixtures/mined-ground-truth.json
python3 tools/bugtrail/cli.py --eval --manifest fixtures/mined-ground-truth.json --repo .
```

Expect: 2 bugs derived, `2/2`. This proves the *mechanism*, not accuracy — see [§6](#6-what-we-proved).

4.5 Unit tests

BASH

```
python3 -m unittest discover -s tools/bugtrail/tests -v
```

Expect: 15 passing, including the CodeSage format contract test.

4.6 Machine-readable output

BASH

```
python3 tools/bugtrail/cli.py --comment fixtures/triage/SYN-001.txt --json
```

Expect: JSON with suspects, commit SHAs, authors, scores, reasons. This is what a Jira adapter will consume.

4.7 The Cursor skill

In a Cursor chat inside this repo:

```
use bugtrail on fixtures/triage/SYN-001.txt
```

****Expect:**** the agent reads ``.cursor/skills/bugtrail/SKILL.md``, runs the engine, fills the test assertion, verifies it, and composes the report.

4.8 Regenerating the fixture from scratch

```
git reset --hard origin/main
./tools/seed_fixture.sh
```

BASH

⚠ Do **not** run `git clean -fd` — it deletes untracked work.

4.9 Useful flags

FLAG	PURPOSE
<code>--repo <path></code>	Analyse a different repository
<code>--history-limit <n></code>	Commits inspected per file; lower on large repos
<code>--no-module-expansion</code>	Consider only the seed file, not siblings
<code>--reported-at <iso></code>	Override the report timestamp
<code>--manifest <path></code>	Score against a different bug set
<code>--config <path></code>	Alternative tuning

5. How it works

5.1 Pipeline

```

CodeSage triage comment (file today, Jira later)
|
codesage.py | ADF flatten → regex parse → fail closed if no "File:"
▼
TriageInput (normalised seed: file, summary, platform, team)
|
archaeology.py | — git only, no model —
| • history via --follow (survives renames)
| • module siblings
| • commit → PR: squash "(#N)" | merge "Merge pull request #N"
| | ancestry-path walk | none
| • exclusions: bot, revert, generated, post-report
| • diff analysis: substantive? rename? keywords? symbols?
▼
ranking.py | four weighted signals → penalties → one entry per PR
▼
codeowners.py → report.py ← testgen.py
owning team      full report      framework-correct scaffold

```

5.2 Commit → PR mapping

Three cases, all handled:

MERGE STYLE	COMMIT SUBJECT	HOW IT IS RESOLVED
Squash merge	Fix preroll propagation (#11)	Regex on the subject
Merge commit	Merge pull request #10 from ...	Regex on the subject
Merged <i>via</i> a merge commit	refactor: rename ... (no number)	Walk --ancestry-path forward to the earliest containing merge
Pushed straight to main	hotfix: widen tolerance	No PR — reported as a bare commit

5.3 Scoring

SIGNAL	WEIGHT	WHY
Recency	0.35	Exponential decay toward the report date; regressions are usually recent
Keyword overlap	0.35	Matched against code lines only , so comments cannot inflate it
Seed-file match	0.20	The file CodeSage named outranks a module sibling
Substantiveness	0.10	Did the change touch code at all?

Then **penalties**: comment-only and rename-only changes are multiplied by `1 - cosmeticPenalty` (default `0.75`).

This penalty does the real work. In the fixture, a docs-only commit lands on the culprit file *one day later* than the true cause. Pure recency ranks it first and gets the wrong answer.

Finally: collapse to one entry per PR, and if the top score is below `minConfidence`, report **no suspect** rather than guessing.

5.4 Configuration

Everything is in `tools/bugtrail/config.json` — never in code, so another team adopts this by editing a file:

```
{
  "historyLimit": 60,
  "expandToModule": true,
  "halfLifeDays": 30,
  "moduleWeight": 0.5,
  "cosmeticPenalty": 0.75,
  "minConfidence": 0.35,
  "maxSuspects": 3,
  "excludeReverts": true,
  "weights": {
    "recency": 0.35,
    "keyword": 0.35,
    "seedFile": 0.20,
    "substantive": 0.10
  },
  "botAuthorPatterns": ["\\[bot\\]", "^dependabot", "^github-actions", "^svc-"],
  "generatedPathGlobs": ["*/Generated/*", "*/generated/*"],
  "generatedMarkers": ["GENERATED CODE DO NOT MODIFY", "@generated"]
}
```

JSON

6. What we proved

6.1 The fixture is adversarial by design

The seeded repository contains deliberate traps, so a naive implementation fails:

TRAP	WHAT IT DEFEATS
Comment-only change to the culprit file, one day later	"Most recent commit wins"
A file rename	Diffs that look like whole-file rewrites
A culprit predating the rename	Any approach without <code>git log --follow</code>
A revert commit	Blaming the person who undid something
A <code>github-actions[bot]</code> commit	Blaming automation
A file marked <code>GENERATED CODE DO NOT MODIFY</code>	Blaming whoever ran a generator
A hotfix pushed straight to main, no PR	Assuming every commit has a PR

6.2 Results

CHECK	RESULT
Fixture eval	<code>precision@1: 3/3</code> , <code>precision@3: 3/3</code>
SZZ-derived eval	<code>2/2</code> (mechanism proof only — see below)
Unit tests	15 passing
Attribution proof	Test fails at HEAD, passes before suspect commit → <code>CONFIRMED</code>

6.3 Read these numbers honestly

`3/3` is a regression guard, not evidence of accuracy. Three cases, hand-built, scored by a ranker tuned against them. It exists so a change to the ranker cannot silently make attribution worse. `2/2` from SZZ is a mechanism demo. This repository's history only yields two mineable fixes, and in both the blamed cause is the file's *creation* commit — an artefact of a short history. It shows the pipeline runs; it measures nothing.

A meaningful figure needs a repository with years of history and enough single-file fixes for the sample size to be worth quoting. The harness is ready for that: `--eval --manifest <file> --repo <path>`.

7. Key design decisions

7.1 Keep the model out of the load-bearing path

DETERMINISTIC — REPRODUCIBLE FROM THE REPO ALONE	JUDGEMENT — MODEL OR HUMAN
Which commits touched the file	Filling a regression test's assertion
Which PR each commit came from	Phrasing the summary for humans
Exclusions (bots, reverts, generated)	Deciding whether a suspect is plausible
The scores and their ranking	Explaining <i>why</i> a change broke behaviour

Why: any suspect can be re-derived by hand with `git log`, and any score recomputed from the printed reasons. Nothing has to be taken on trust — which is what makes the output survive review.

A side effect worth stating: swapping model or vendor **cannot change which PR is named**.

7.2 Fail closed, never guess

- No `- File:` line in the comment → return `None`, analyse nothing.
- Top score below threshold → report **no suspect**.
- Seed file is generated → refuse to attribute.

A wrong seed produces *confidently* wrong attribution, which is worse than no answer.

7.3 The drafted test's assertion is deliberately a `TODO`

The scaffold is deterministic: path, framework, class name, and the symbol under test are all derived from the suspect diff. The assertion is not, because writing it requires knowing what the code was *supposed* to do.

A generator that emitted confident-looking assertions would demo beautifully and mislead on the first real bug.

7.4 Attribution is a suggestion, not a verdict

Every report says *"likely related changes"* and ends with *"ranked suggestions, not attributions of fault."* Attribution errors are cheap; blaming the wrong engineer is not.

8. Bugs we hit and fixed

Both only appear on realistic repositories — worth knowing for the Q&A.

8.1 Renames faked keyword matches

Symptom: the rename PR outranked the true culprit.

Cause: git renders a pure rename as a whole-file delete plus add, so the rename commit appeared to mention every keyword in the bug report.

Fix: detect renames with `--name-status -M`, run **without** a pathspec (rename detection needs both sides of the diff), and treat them as non-behavioural.

8.2 Historical paths broke diff analysis

Symptom: the true culprit vanished from the results entirely for the renamed-file case.

Cause: `--follow` correctly walked back past the rename, but the engine then diffed that old commit against the file's **current** name — which did not exist yet. Empty diff → looked cosmetic → penalised out.

Fix: parse `--name-status` alongside the log to recover each commit's path *at that commit*.

This is the more instructive failure: invisible on a repository where nothing has moved, guaranteed on one where things have.

8.3 Timestamp parsing

Real histories mix `+00:00` and `Z` offsets. Python 3.9's `fromisoformat` rejects the latter and crashed the miner. Now handled centrally in `parse_timestamp`.

9. Repository layout

```
.
├─ demo.sh                # ← the 3-second end-to-end demo
├─ BUGTRAIL.md            # ← this document
├─ docs/BUGTRAIL.pdf      # ← this document, printable
├─
├─ .cursor/skills/bugtrail/
│   └─ SKILL.md           # Cursor skill wrapping the workflow
├─
├─ tools/bugtrail/
│   ├── cli.py             (330)    # sources, orchestration, eval
│   ├── archaeology.py     (389)    # git history, PR mapping, exclusions
│   ├── mine_szz.py        (269)    # derive ground truth from history
│   └── report.py          (171)    # full report rendering
```

```

|   ├── codesage.py          (140)    # ADF flatten + comment parsing
|   ├── testgen.py          (137)    # regression test scaffolding
|   ├── ranking.py          (123)    # keyword extraction + scoring
|   ├── models.py           (82)     # shared data types
|   ├── codeowners.py       (75)     # owner resolution
|   ├── config.json         # all tuning lives here
|   ├── README.md           # setup, usage, limitations
|   ├── ARCHITECTURE.md     # design and data flow
|   ├── tests/test_bugtrail.py (215)  # 15 tests
|   └── verify/
|       ├── verify.sh       # compiles assertion at two revisions
|       └── main.swift      # the assertion itself
|
|   ├── fixtures/
|       ├── ground-truth.json # answer key for the seeded bugs
|       └── triage/SYN-00{1,2,3}.txt # synthetic CodeSage comments
|
|   ├── tools/docs/md2pdf.py (330)    # renders this document to PDF
|   └── tools/seed_fixture.sh # regenerates the adversarial history

```

Regenerate the PDF after editing this file:

```
python3 tools/docs/md2pdf.py BUGTRAIL.md docs/BUGTRAIL.pdf
```

BASH

~1,930 lines of Python, standard library only.

10. Path to production — Jira

This is the remaining work. The architecture was built for it: the engine never learns where its input came from or where its output goes.

```

TriageSource → yields TriageInput
├── codesage_source (built) reads a comment file
└── JiraSource      (TODO)  reads issue + CodeSage comment

ResultSink ← receives ranked suspects
├── ConsoleSink    (built) renders the report
└── JiraCommentSink (TODO) posts one idempotent comment

```

Adding Jira is a **new adapter**, not a change to archaeology or ranking.

10.1 What is already done

PIECE	STATUS
ADF (Atlassian Document Format) flattening	✓ <code>codesage.flatten_adf()</code> — Jira v3 returns comment bodies as ADF JSON, not text
CodeSage comment parsing	✓ <code>codesage.parse_adf_comment()</code>
Structured output for posting	✓ <code>cli.to_dict()</code> / <code>--json</code>
Report shaped like a Jira comment	✓ <code>report.render_report()</code>
Platform → repo routing	✓ derived from the seed file extension

10.2 Step 1 — Trigger

Recommended: a JQL poller, scoped by an opt-in label. No infrastructure, no admin rights, and every team decides for itself whether it wants BugTrail.

```
labels = bugtrail
AND issuetype in ("Bug")
AND created >= -30d
ORDER BY created DESC
```

SQL

That is exactly what `jira_agent.py --label bugtrail` builds. Three properties matter:

- **Opt-in, not opt-out.** A bug under *any* story in *any* project joins the moment someone adds the label, and leaves when they remove it. Widening the scope needs no redeploy; nobody is enrolled without asking.
- **Bugs only.** Attribution is meaningless for a Story or a Task — they describe work to do, so no past change "caused" them. The type filter is applied in the JQL and re-checked per issue, because `--jql` is caller-supplied and a comment on the wrong ticket cannot be taken back.
- **An age floor.** Without `created >= -30d`, one label typo on an old ticket drags the back catalogue into scope on the first run.

Hackathon scope. During the hackathon every demo bug is filed as a Sub-task of one story, PLAY-126471:

```
python3 tools/bugtrail/jira_agent.py --parent PLAY-126471 \
--issue-types "Sub-task" --post --once
```

BASH

Sub-tasks rather than Bugs, because PLAY only allows a Sub-task to be created directly under a Story — a Bug sits at the same hierarchy level as a Story, and its create screen additionally requires an Epic and a

Team. Filing demo bugs as Sub-tasks avoids all of that, and a reporter can raise one in seconds.

The agent refuses to accept Sub-tasks unless `--parent` or `--label` is also given. A Bug is a defect wherever it lives, but most sub-tasks are ordinary planned work, so accepting them project-wide would mean commenting on work that was never broken. Pinning them to one story keeps that safe. **Production stays Bug only** — this widening exists solely because the hackathon has to file demo tickets inside a live project.

OPTION	INFRASTRUCTURE	ADMIN RIGHTS	NOTES
JQL poller	A machine of yours, awake	None	Idempotent; a missed tick is picked up on the next one
Automation → GitHub Actions ✓	None of ours	Project admin	Fires on creation; GitHub supplies the compute — §10.8
Resident GitHub Actions watcher ✓	None of ours	None	15s response with no admin rights at all — §10.8

Automation and the script are not alternatives. A Jira Automation rule cannot clone a repository or walk git history; all it can do is fire a *Send web request*. So Automation is always the trigger and never the worker. What it can call, though, is GitHub's own API: a `repository_dispatch` starts a workflow that does the work on GitHub's runners, so "a reachable endpoint" stops meaning "a server we operate". That is the deployed shape — see §10.8. The poller remains useful as a demo and as a backstop, and the scoping and comment logic are identical either way.

Licensing is not a blocker. The REST API is available on every Jira plan. Only *native Automation rules* are metered.

10.3 Step 2 — Read path

```
# jira_source.py (to build)
issue    = get_jira_issue(key)                # summary, created, comments
comment  = first(c for c in issue.comments
                if c.author.accountId == CODESAGE_ACCOUNT_ID)
triage   = parse_adf_comment(comment.body)     # already implemented
```

PYTHON

Identify CodeSage by service account, not by text. The account is `svc-wbdstreaming-play-codesage`; matching on the author id is far more robust than grepping for a marker string. Use JQL to *find* candidates, then verify the author id when fetching.

10.4 Step 3 — Write path

```
# jira_bot.py
MARKER = "bug-slayers:v1"

# Comments written before the bot was renamed carry the old token, and the
# marker is the only way a re-run recognises its own work. Drop it and every
# ticket already answered looks untouched, so it gets answered twice.
LEGACY_MARKERS = ("bugtrail:v1",)

existing = find_comment_matching(issue, (MARKER,) + LEGACY_MARKERS)
body     = render_comment(result) + "\n\n%s_" % MARKER

if existing:
    update_comment(issue, existing.id, body)  # never duplicate
else:
    add_comment(issue, body)
```

Rules:

- **One comment per issue.** Update in place; a re-run must never post a second comment.
- **Never retire a marker.** Recognising the old one is what makes renaming the bot safe.
- **Only post above the confidence threshold.** Below it, post nothing.
- **Keep the disclaimer.** Mirror CodeSage's own "please review" convention.
- **Never @-mention** an author below high confidence.

10.5 Step 4 — Repository routing

```
REPOS = {
    "ios":      "/path/to/ios-repo",
    "android":  "/path/to/android-repo",
}

repo = REPOS[triage.platform]  # platform derived from the seed file extension
```

Clones must be kept fresh (`git fetch` before analysis) or attribution will silently miss recent PRs.

10.6 Step 5 — Let the agent fill the test assertion

Behind the Cursor skill, gated on human review. Never auto-commit a generated test.

10.7 Step 6 — Actually running it

Nothing comments on a ticket until a process is running somewhere. The comment PLAY-126480 received from *Automation for Jira* is a native rule posting fixed text — it needs no compute, so Jira can host it.

BugTrail clones a repository and walks its history, so it needs a machine.

Credentials, once. Create a token at id.atlassian.com and keep it out of the repository:

BASH

```
mkdir -p ~/.config/bugtrail
cat > ~/.config/bugtrail/env <<'EOF'
export JIRA_BASE_URL=https://wbdstreaming.atlassian.net
export JIRA_EMAIL=you@wbd.com
export JIRA_API_TOKEN=paste-token-here
EOF
chmod 600 ~/.config/bugtrail/env
```

Three ways to run it, in increasing order of permanence:

	HOW	LATENCY	NEEDS
Terminal	<code>jira_agent.py ... --watch</code>	Poll interval	Nothing. Best for a live demo
GitHub Actions, scheduled	<code>.github/workflows/bugtrail.yml</code>	~5 minutes	Three repository secrets
GitHub Actions, resident	Chained <code>watch-cycle</code> runs, restarted by the sweep	15 seconds	Three secrets and <code>CHAIN_TOKEN</code>
GitHub Actions, on request	<code>gh workflow run BugTrail -f watch_minutes=20</code>	15 seconds, for the window you ask for	Three repository secrets

BASH

```
# demo: poll every 15s in the foreground, so the audience sees it happen
python3 tools/bugtrail/jira_agent.py --parent PLAY-126471 \
  --issue-types "Sub-task" --post --interval 15
```

`--interval` implies `--watch`. It used to require both, and asking for an interval without it produced one healthy-looking sweep and a silent exit — the worst failure mode available, since a watcher that has stopped looks exactly like a watcher with nothing to do.

How the resident watcher stays resident. A job cannot outlive its runner, so "permanent" is a chain: each cycle polls for 55 minutes and dispatches a successor before it ends. Run once, that arrangement is fragile, and it failed in production in two distinct ways worth recording.

The first is that nothing owned the question of whether a watcher existed. The chain was started once by hand, so when GitHub declined to give a cycle a runner — one sat queued for 45 minutes — the chain simply ended. Nothing reported it, because a watcher that has stalled looks exactly like a watcher with nothing to do; the only symptom was tickets being answered late by the sweep. The five-minute sweep now checks whether a cycle is alive and starts one if not, which is what makes the chain self-healing: the worst case is a five-minute gap rather than an outage lasting until somebody notices.

That check counts a run as alive only if it started in the last 70 minutes. GitHub occasionally leaves a run reporting `in_progress` after its jobs have finished — one such ghost is in this repository's history, uncancellable, with the API returning HTTP 500 on both `cancel` and `force-cancel`. Without the age window a single ghost would look like a healthy watcher forever and suppress every restart.

The second failure was duplication. Handover ran on `always()`, which includes cancellation, and the only thing that cancels a cycle is its own replacement arriving through the concurrency group — so each cancellation left two chains instead of one. Two pollers can both read a ticket as unanswered and comment on it twice. Handover is now gated on `!cancelled()`.

A macOS `launchd` service used to be the unattended option. It was removed because it was strictly worse on the thing that matters: it answered tickets only while one particular Mac was awake. It also needed the tool copied to `~/local/share/bugtrail`, since macOS gates `~/Documents` behind per-application consent and a `launchd` job inherits none of the access your terminal was granted — pointing it at a checkout there fails with `Operation not permitted` indefinitely. Worth knowing if you ever reintroduce it; `git log -- deploy/` has the working version.

10.8 Step 7 — Triggering from Jira, with nothing running locally

Every option above needs a machine of yours to be awake. A reporter filing a bug at 2am from another timezone should get the same answer as one filing it while you have a terminal open, so the trigger belongs in Jira and the compute belongs somewhere that is always on.

A Jira Automation rule fires a `repository_dispatch` at GitHub the moment a work item is created, and the workflow does the triage on GitHub's runners. Measured end to end, the comment lands about 20–30 seconds after *Create*, most of which is GitHub starting a runner.

First, the repository secrets the runner needs:

BASH

```
gh secret set JIRA_BASE_URL --repo <owner>/<repo> # https://wbdstreaming.atlassian.net
gh secret set JIRA_EMAIL --repo <owner>/<repo>
gh secret set JIRA_API_TOKEN --repo <owner>/<repo>
```

Then a GitHub token for Jira to authenticate with — a fine-grained token scoped to this one repository with **Contents: Read and write**, which is what `repository_dispatch` requires. Then in **Project settings → Automation → Create rule**:

FIELD	VALUE
When	Work item created
If	Issue Type equals Sub-task, parent = PLAY-126471
Then	Send web request
URL	<code>https://api.github.com/repos/<owner>/<repo>/dispatches</code>
Method	POST
Headers	<code>Authorization: Bearer <github-token></code> , <code>Accept: application/vnd.github+json</code>
Body	custom data — <code>{"event_type": "jira-issue", "client_payload": {"key": "{{issue.key}}"} }</code>

The rule sends nothing but the issue key, and the workflow treats it as a hint rather than an instruction. The key is validated against `^[A-Z][A-Z0-9]*-[0-9]+$` and passed through the environment rather than interpolated into the shell, because `{{{ }}` pasted into a `run:` block is executed before it is ever a string. The issue is then read back from Jira with our own credentials and filtered through the same `--parent` scope the sweep uses, so naming a ticket over the API does not put it in scope — a dispatch for an unrelated ticket resolves to `0 issue(s) in scope`.

The 15-minute schedule stays enabled underneath as a backstop, covering a disabled rule, a GitHub incident, or an item moved into scope after creation. Because commenting is idempotent, the sweep re-reading an item the rule already handled is a no-op rather than a duplicate.

The scheduled options run `--once` per tick rather than looping. A crashed loop stays dead until somebody notices; a scheduler just runs again on the next tick.

Self-hosting the receiver instead. A `jira_webhook.py` existed for the case where you would rather Jira called our code directly — no GitHub in the path, single-digit-second latency. It was removed rather than kept as an option, because it was never deployed: it needs a public HTTPS endpoint that somebody keeps running, and the dispatch route above gets within seconds of the same latency while requiring no server of ours at all. Keeping two receivers meant two authentication paths to reason about for one behaviour. `git log -- tools/bugtrail/jira_webhook.py` has it if the trade ever changes.

This is the one option that needs **project admin rights** to configure, and a URL Jira can reach. A tunnel is fine for a demo; production would host the same script behind a real endpoint.

It will comment as you. A personal API token carries your identity, so tickets will show *Monish K* rather than a bot. CodeSage avoids this with a service account (`svc-wbdstreaming-play-codesage`); getting an equivalent for BugTrail is an IT request, and it is the honest answer to "is this a bot or a person?" during a demo.

11. Production hardening checklist

11.1 Security and IP

REQUIREMENT	STATUS	ACTION
No PII, tokens, or credentials in prompts	⚠️	Write a sanitiser with its own tests before any log or stack trace reaches a model
Service account, least privilege	❌	Jira: <i>add comment</i> only. GitHub: <i>read</i> only. Never a personal token
Secrets in the approved vault	❌	Not env files, never in prompts
Audit trail	❌	Log what was read, what was decided, what was posted
No unvetted dependencies	✅	Standard library only
Nothing internal in public repos	⚠️	Repository is public . Every fixture is synthetic and <code>fixtures/codesage/</code> is gitignored, but this document does describe CodeSage's behaviour and comment format — worth a review before wider circulation
Human review before code lands	✅	Drafted tests are proposals, never committed

11.2 Reliability

CONCERN	HANDLING
Duplicate comments	Marker + update-in-place
Jira or GitHub unavailable	Bounded retries with backoff; degrade to a partial comment rather than failing loudly
CodeSage format changes	Contract test pinned to the sample comment — CI fails loudly instead of parsing to nothing
Runaway posting	Feature flag / kill switch, no deploy required
Stale clones	<code>git fetch</code> before analysis

11.3 Observability and feedback

- **Metrics:** runs, latency, post success rate, error rate, and precision over time.
- **Feedback signal:** a Jira label (`bugtrail-correct` / `bugtrail-wrong`) applied by engineers. This does three jobs at once — collects ground truth continuously, lets you tune ranking weights on real data, and produces the measured-impact number.

11.4 Rollout stages

1. Shadow mode	compute and log, post nothing	← validates safely
2. One project	post to a single project, opt-in	
3. Opt-in teams	teams enable it themselves	
4. Default on	with a documented kill switch	

12. Known limitations

LIMITATION	IMPACT
Keyword matching is substring-based	<code>roll</code> matches <code>preroll</code> — loose but harmless
Module expansion covers only the immediate directory	A cause two directories away is missed
PR mapping relies on GitHub message conventions	A repo with different merge messages needs the GitHub API path
Bugs caused by config, content, or backend	No code culprit exists; the confidence threshold prevents inventing one
Precision unmeasured on a large real corpus	See §6.3
Kotlin test written but not executed	No Gradle here; only the Swift case is mechanically confirmed

13. Work split and next steps

Suggested split — five developers, one repository, one target repo

Scope is deliberately narrowed to a **single target repository**. Multi-repo routing is a day-two concern and buys nothing in a demo.

Each lane owns its own files, so five people can work in one repository without colliding. The interfaces between lanes are the existing dataclasses in `models.py`, which are agreed in the first half hour and then frozen.

LANE	OWNER	OWNS THESE FILES	DELIVERS
1. Core + integration	Monish	<code>models.py</code> , <code>config.json</code> , <code>cli.py</code> , <code>report.py</code>	Reviews and merges every PR, keeps <code>main</code> green, owns the demo script
2. Jira read + sanitiser	Dev 2	<code>jira/source.py</code> , <code>sanitiser.py</code>	Issue + CodeSage comment via the approved Atlassian integration, returning a <code>TriageInput</code> . Redaction happens here, at the boundary where external text enters
3. Jira write	Dev 3	<code>jira/sink.py</code>	Posts the report as a comment, with the marker and update-in-place so re-runs never duplicate
4. Scheduling + audit	Dev 4	<code>jira/poller.py</code> , <code>audit.py</code>	JQL poller, shadow mode (log, do not post), secrets loading, and a log of what was read, decided, and posted
5. Eval at scale	Dev 5	<code>eval/</code>	Runs <code>mine_szz.py</code> over a large real repository and reports precision@1 and @3. Needs no Jira access, so it starts immediately and cannot be blocked

Rules that keep five people out of each other's way

- **Only lane 1 edits the shared files.** Everyone else opens an issue or asks;

a drive-by edit to `models.py` or `config.json` is what causes the painful conflict.

- **Freeze the interfaces first, then build against fakes.** Lanes 3 and 4 should

never wait on lane 2 — a stub `TriageInput` unblocks them on day one.

- **Merge small and often.** No branch older than half a day.
- **Shadow mode until the very end.** Nothing posts to a real ticket until the

read path, the sanitiser, and idempotency are all in and reviewed.

Priority order

1. **Real eval on a large repository** — the number that makes everything else believable
2. **Jira read path** in shadow mode, with the sanitiser in front of it
3. **Jira write path** with idempotency
4. **JQL poller**
5. **Feedback label** to collect ground truth continuously

If time runs short, land these three

The eval number, the read path, and a report printed to the console. A believable precision figure with a manual paste into Jira demos far better than a fully automated loop whose accuracy nobody can vouch for.

The 30-second summary for a reviewer

CodeSage tells us *where* to look. BugTrail closes the loop to source control: it names the pull requests that likely caused the bug, who authored them, which team owns the file, and it drafts a regression test — then **compiles that test at two revisions to prove the behaviour changed at that commit and nowhere else**. The part that decides *which PR* is fully deterministic, so every result can be reproduced by hand with `git log`. A model is used only where judgement is genuinely required, and never for attribution.